

Computing Platform for Autonomous Driving (CP4AD)

L3. RTOS #3

Prof. Jong-Chan Kim

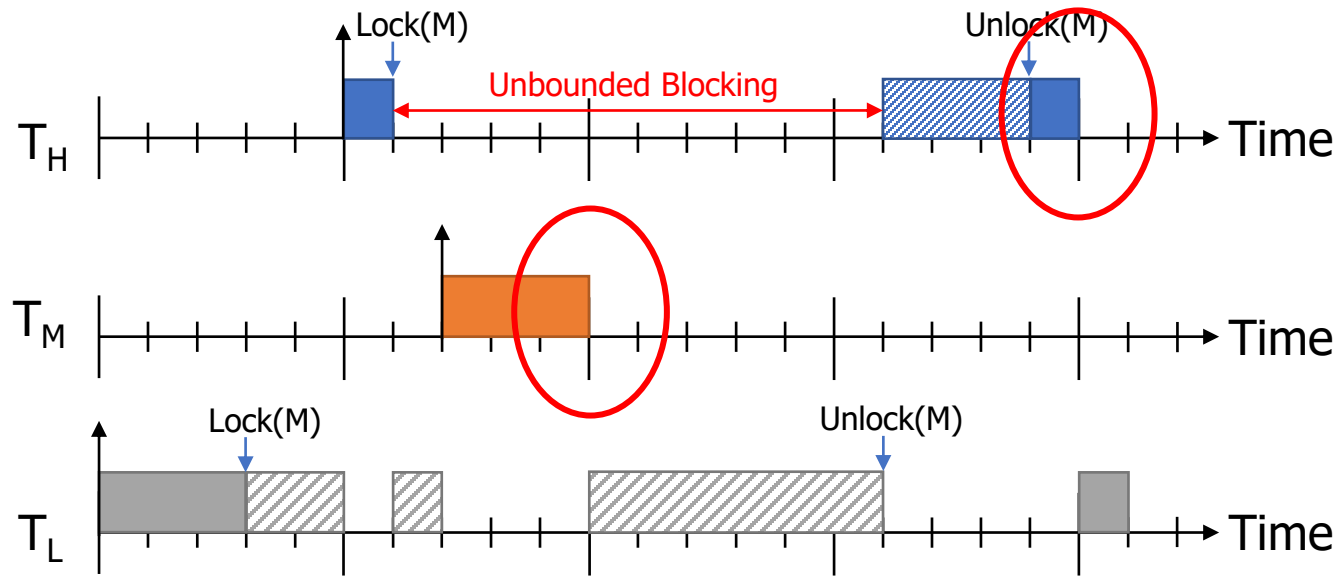
Dept. Automobile and IT Convergence



국민대학교
KOOKMIN UNIVERSITY

Review. Priority Inversion

- Lock에 의해 T_H 가 T_M 보다 늦게 종료



```
Monitor Mode Serial View Mode Text Port COM9 - Arduino Uno(COM9)
---- Opened the serial port COM9 ----

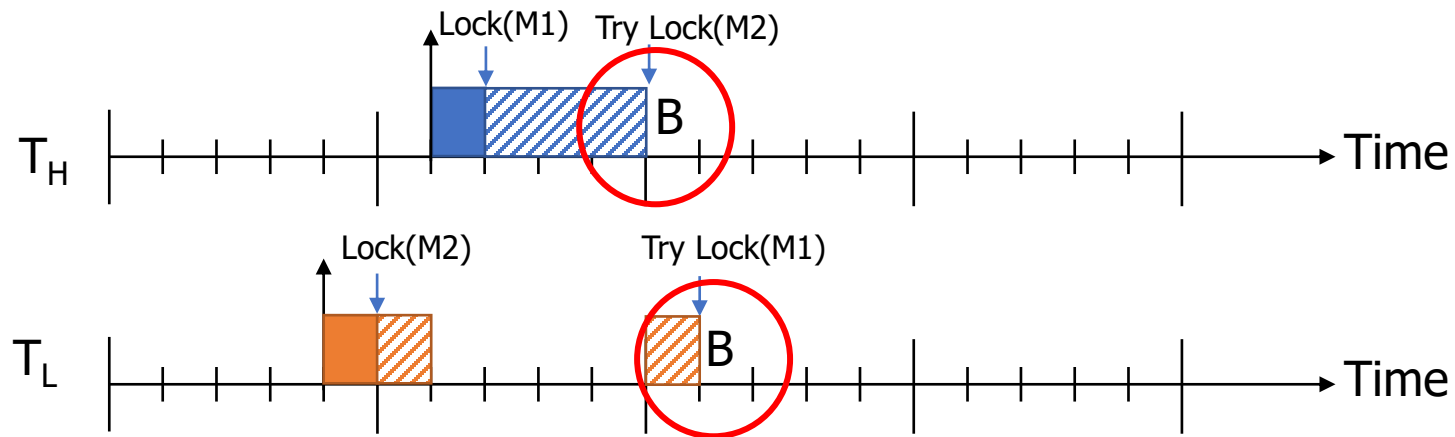
.....
...OS Starts...
.....

-4:
-3:
-2:
-1:
0: <TaskL begins.>
1:
2:
3: TaskL : Try Lock(M1). TaskL : Get Lock(M1).
4:
5: <TaskH begins.>
6: TaskH : Try Lock(M1). --> BLock
7: <TaskM begins.>
8:
9:
10: <TaskM ends.>
11:
12:
13:
14:
15:
16: TaskL : Release Lock(M1). TaskH : Get Lock(M1).
17:
18:
19: TaskH : Release Lock(M1).
20: <TaskH ends.>
21: <TaskL ends.>
22:
23:
---- Closed the serial port COM9 ----
```

Unbounded Blocking

Review. Deadlock

- 두 task가 서로 Lock 해제를 대기



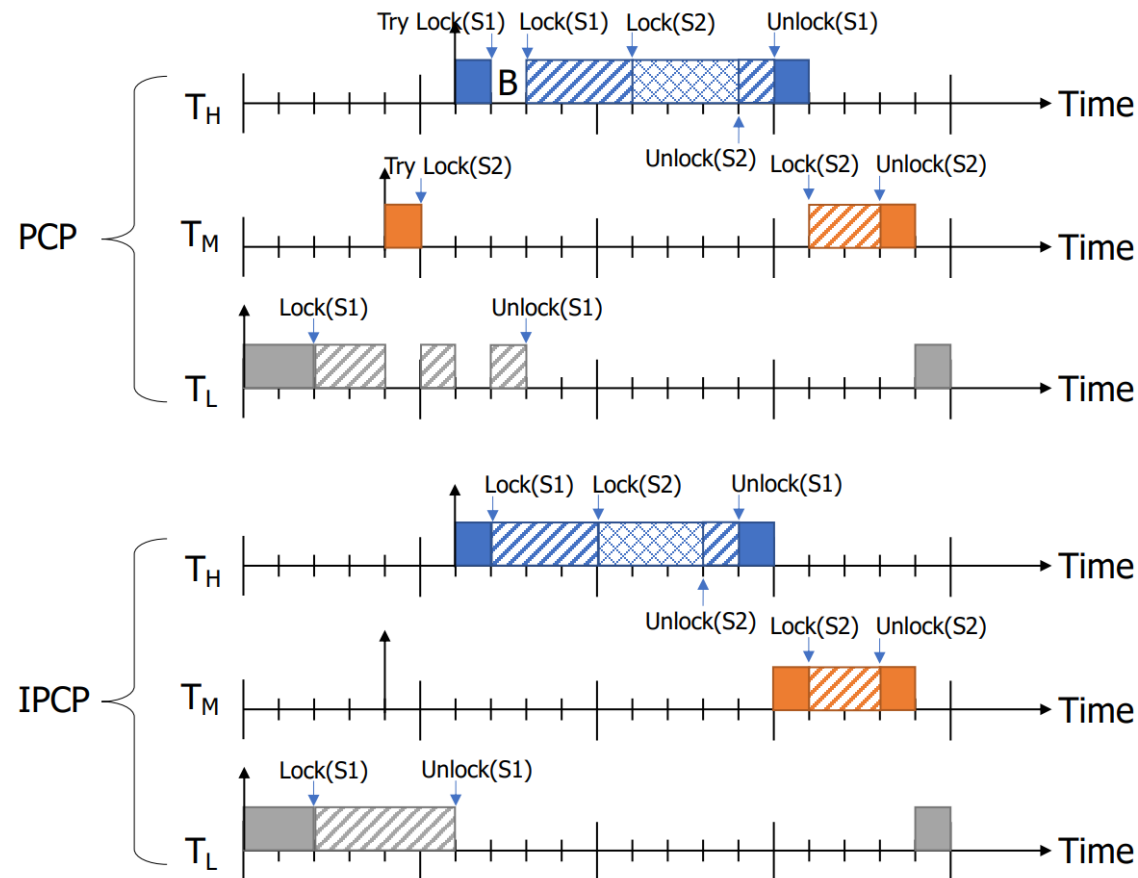
```
Monitor Mode Serial View Mode Text Port COM9 - Ardui
---- Opened the serial port COM9 ----

.....
...OS Starts...
.....

-4:
-3:
-2:
-1:
0:
1:
2:
3:
4: <TaskL begins.>
5: TaskL : Try Lock(M2). TaskL : Get Lock(M2).
6: <TaskH begins.>
7: TaskH : Try Lock(M1). TaskH : Get Lock(M1).
8:
9:
10: TaskH : Try Lock(M2). --> BLock
11: TaskL : Try Lock(M1). --> BLock
12:
13:      Deadlock
14:
15:
---- Closed the serial port COM9 ----
```

12-1. IPCP

- IPCP (Immediate Priority Ceiling Protocol)
 - When locking a resource, the task's priority is immediately raised to the ceiling of the lock

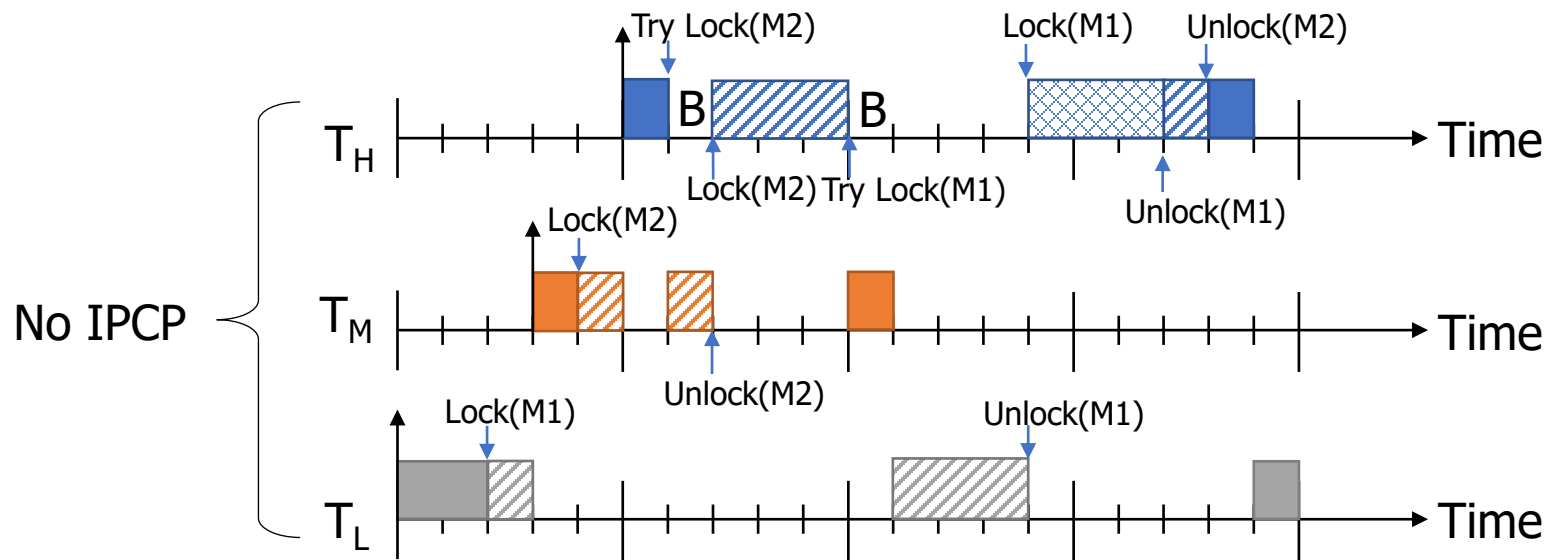


T_L 's priority is **immediately** increased upon acquiring the lock

12-1. IPCP

• Mutex로 아래 시스템 구현

1. asw.c 파일만 수정
2. 그림 기반으로 10번 실습 되짚어가며 진행

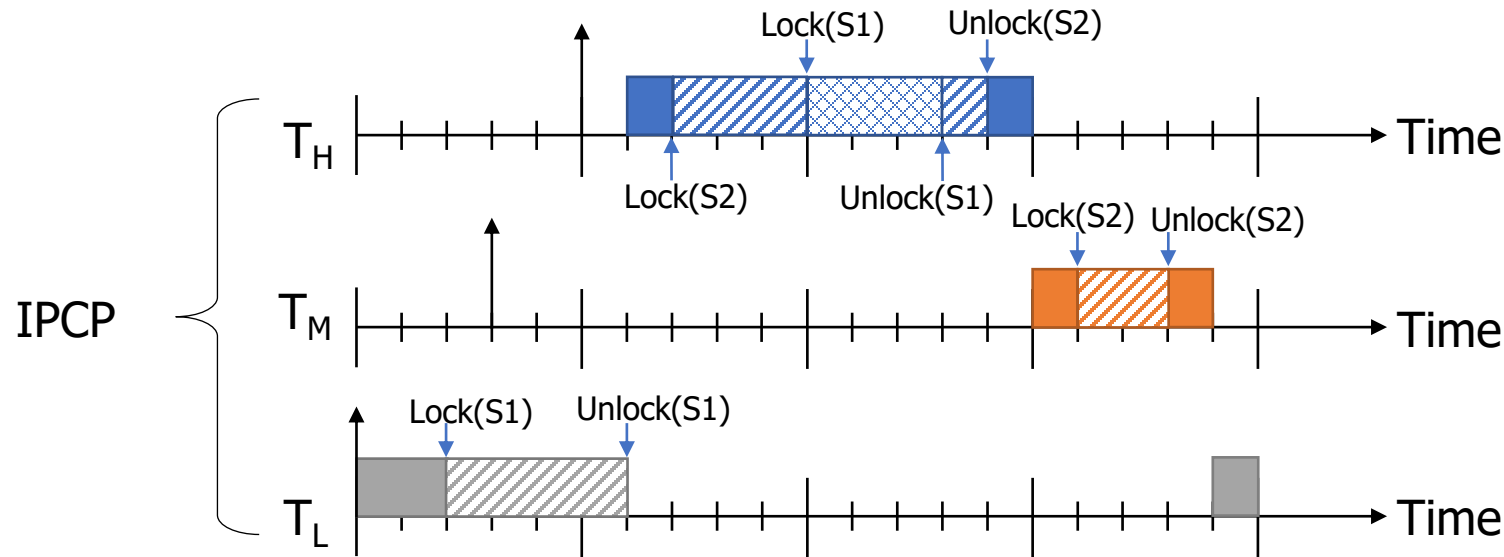


```
0: <TaskL begins.>
1: TaskL : Try Lock(S1). TaskL : Get Lock(S1).
2: <TaskM begins.>
3: TaskM : Try Lock(S2). TaskM : Get Lock(S2).
4: <TaskH begins.>
5: TaskH : Try Lock(S2). --> BLock
6: TaskM : Release Lock(S2). TaskH : Get Lock(S2).
7:
8:
9:
10: TaskH : Try Lock(S1). --> BLock
11: <TaskM ends.>
12:
13:
14: TaskL : Release Lock(S1). TaskH : Get Lock(S1).
15:
16:
17: TaskH : Release Lock(S1).
18: TaskH : Release Lock(S2).
19: <TaskH ends.>
20: <TaskL ends.>
```

12-1. IPCP

- Mutex를 Resource로 교체, 차이 비교

1. asw.c 파일만 수정
2. 그림 기반으로 10번 실습 되짚어가며 진행

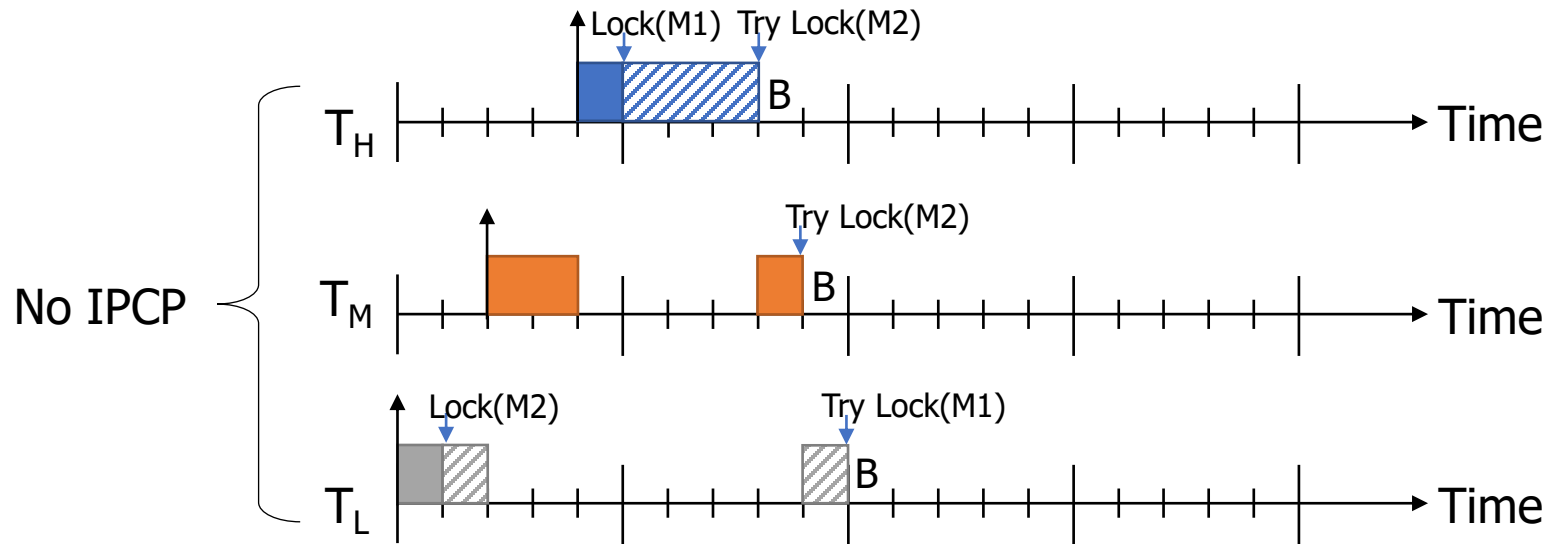


```
0: <TaskL begins.>
1:
2: TaskL : Try Lock(S1). TaskL : Get Lock(S1).
3:
4:
5:
6: TaskL : Release Lock(S1). <TaskH begins.>
7: TaskH : Try Lock(S2). TaskH : Get Lock(S2).
8:
9:
10: TaskH : Try Lock(S1). TaskH : Get Lock(S1).
11:
12:
13: TaskH : Release Lock(S1).
14: TaskH : Release Lock(S2).
15: <TaskH ends.> <TaskM begins.>
16: TaskM : Try Lock(S2). TaskM : Get Lock(S2).
17:
18: TaskM : Release Lock(S2).
19: <TaskM ends.>
20: <TaskL ends.>
```

12-2. IPCP

- Mutex로 아래 시스템 구현

1. asw.c 파일만 수정
2. 그림 기반으로 10번 실행 되짚어가며 진행

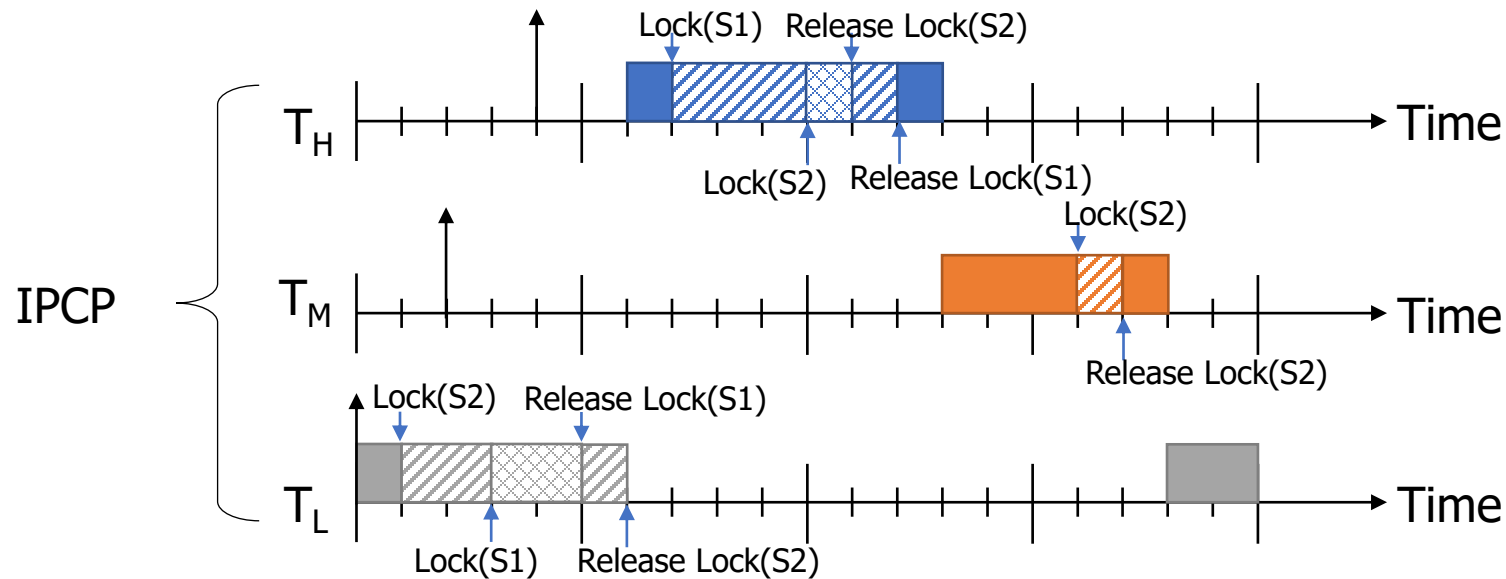


```
0: <TaskL begins.>
1: TaskL : Try Lock(S2). TaskL : Get Lock(S2).
2: <TaskM Begins.>
3:
4: <TaskH begins.>
5: TaskH : Try Lock(S1). TaskH : Get Lock(S1).
6:
7:
8: TaskH : Try Lock(S2). --> BLock
9: TaskM : Try Lock(S2), --> BLock
10: TaskL : Try Lock(S1). --> BLock
11:
12:
13:
14:
15:
16:
17:
18:
19:
20:
```

12-2. IPCP

- Mutex를 Resource로 교체, 차이 비교

1. asw.c 파일만 수정
2. 그림 기반으로 10번 실습 되짚어가며 진행



```
0: <TaskL begins.>
1: TaskL : Try Lock(S2). TaskL : Get Lock(S2).
2:
3: TaskL : Try Lock(S1). TaskL : Get Lock(S1).
4:
5: TaskL : Release Lock(S1).
6: TaskL : Release Lock(S2). <TaskH begins.>
7: TaskH : Try Lock(S1). TaskH : Get Lock(S1).
8:
9:
10: TaskH : Try Lock(S2). TaskH : Get Lock(S2).
11: TaskH : Release Lock(S2).
12: TaskH : Release Lock(S1).
13: <TaskH ends.> <TaskM Begins.>
14:
15:
16: TaskM : Try Lock(S2), TaskM : Get Lock(S2).
17: TaskL : Release Lock(S2).
18: <TaskM ends.>
19:
20: <TaskL ends.>
```


Alarm 기반 Activation의 문제점

- OSEK에서 Periodic Task 작성 방법
 - Counter와 연결된 Alarm에서 Activate하도록 OIL 설정
- 문제점
 - Alarm은 런타임에 취소/변경이 가능
 - 실수 혹은 악의적으로 Alarm 변경시 Periodic Task 오동작 가능성
- Example)
 - CancelAlarm() 호출

13.6.3.5 CancelAlarm

Syntax: StatusType CancelAlarm (AlarmType <AlarmID>)

Parameter (In):
 AlarmID Reference to an alarm

Parameter (Out): none

Description: The system service cancels the alarm <AlarmID>.

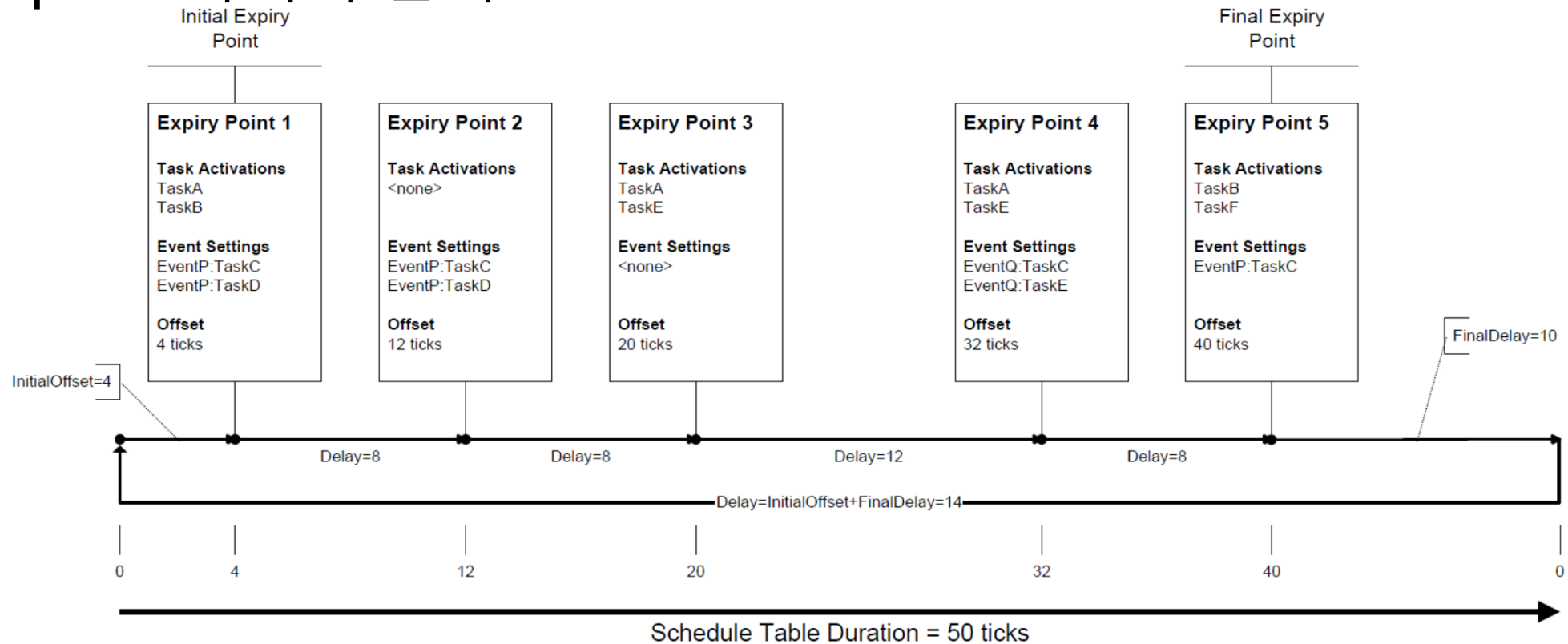
Particularities: Allowed on task level and in ISR, but not in hook routines.
Status:

- Standard:
- No error, E_OK
 - Alarm <AlarmID> not in use, E_OS_NOFUNC
- Extended:
- Alarm <AlarmID> is invalid, E_OS_ID

Conformance: BCC1, BCC2, ECC1, ECC2

Schedule Table Concepts

- Duration: Schedule table의 사이클 타임
- Expiry points: Duration 안에서의 상대 시간 (Activate, SetEvent 가능)
- Initial Offset: 첫 expiry point
- Delay: Expiry point 사이의 간격



13. Schedule Table

```
SCHEDULETABLE SchedTab1 {
```

```
  COUNTER = counter1;
```

```
  DURATION = 10;
```

```
  REPEATING = TRUE;
```

```
  AUTOSTART = TRUE {
```

```
    START_VALUE = 5;
```

```
  };
```

```
  EXPIRE_POINT = ACTION {
```

```
    EXPIRE_VALUE = 0;
```

```
    ACTION = ACTIVATETASK { TASK = TaskH; };
```

```
    ACTION = ACTIVATETASK { TASK = TaskL; };
```

```
  };
```

```
  EXPIRE_POINT = ACTION {
```

```
    EXPIRE_VALUE = 5;
```

```
    ACTION = ACTIVATETASK { TASK = TaskH; };
```

```
    ACTION = SETEVENT { TASK = TaskL; EVENT = Event1; };
```

```
  };
```

```
};
```

Cycle Time

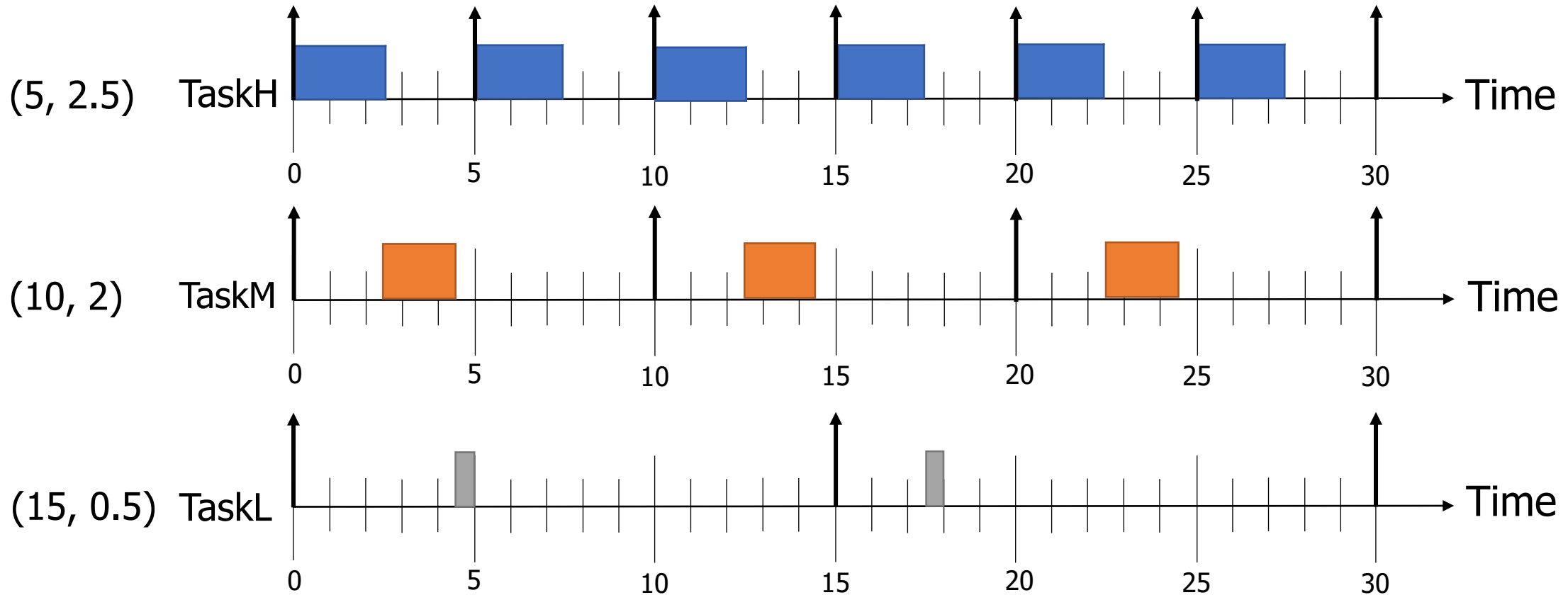
Start Offset

Task Activation

Event Setting

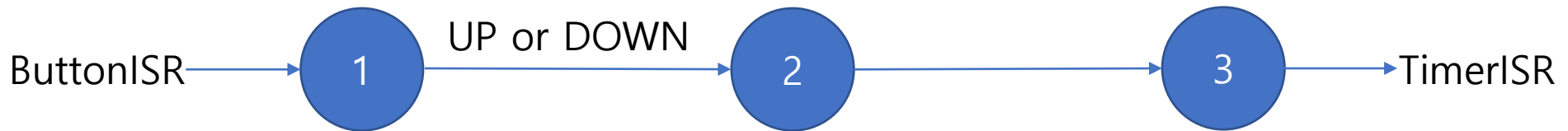
13. Schedule Table

- 3개 Task 생성 및 Scheduling 실험
- Timing diagram



14. End-to-End Delay

- AUTOSAR 기반 DAG (Directed Acyclic Graph) SW 구조
- Sensor에서 Actuator까지 End-to-End Delay 관찰



14. End-to-End Delay

- TASK를 통한 Runnable 실행

asw.c

```
void Runnable_1(void)
{
    R2_in = R1_in;
}
...
ISR2(TimerISR)
{
    static long c = -5;
    char *out = "NA";
    IncrementCounter(counter1);
    if(R3_out == UP)
        out = "UP";
    else if(R3_out == DOWN)
        out = "DN";
    printfSerial("\n4ld: [%s] ", ++c, out);
}
```

asw.c

```
TASK(Task_8s)
{
    printfSerial("Task_8s begins... ");
    Runnable_1();
    Runnable_2();
    Runnable_3();
    printfSerial("Task_8s finishes... ");
    TerminateTask();
}
```

14. End-to-End Delay

- SCHEDULETABLE에 TASK 추가하여 실행

SCHEDULETABLE SchedTab1 {
COUNTER = counter1;
DURATION = 8;
REPEATING = TRUE;
AUTOSTART = TRUE {
 START_VALUE = 5;
};
EXPIRE_POINT = ACTION {
 EXPIRE_VALUE = 0;
 ACTION = ACTIVATETASK { TASK = Task_2s; };
 ACTION = ACTIVATETASK { TASK = Task_4s; };
 ACTION = ACTIVATETASK { TASK = Task_8s; };
};
...
};

14. End-to-End Delay

- End-to-End Delay 차이 존재

```
0: [NA] Task_2s begins... Task_2s finishes... Task_4s begins... Task_4s finishes... Task_8s begins... Task_8s finishes...
1: [NA] <BUTTON UP>
2: [NA] Task_2s begins... Task_2s finishes...
3: [NA]
4: [NA] Task_2s begins... Task_2s finishes... Task_4s begins... Task_4s finishes...
5: [NA]
6: [NA] Task_2s begins... Task_2s finishes...
7: [NA]
8: [NA] Task_2s begins... Task_2s finishes... Task_4s begins... Task_4s finishes... Task_8s begins... Task_8s finishes...
9: [UP]
10: [UP] Task_2s begins... Task_2s finishes...
11: [UP]
12: [UP] Task_2s begins... Task_2s finishes... Task_4s begins... Task_4s finishes...
13: [UP]
14: [UP] Task_2s begins... Task_2s finishes... <BUTTON DOWN>
15: [UP]
16: [UP] Task_2s begins... Task_2s finishes... Task_4s begins... Task_4s finishes... Task_8s begins... Task_8s finishes...
17: [DN]
18: [DN] Task_2s begins... Task_2s finishes...
19: [DN]
20: [DN] Task_2s begins... Task_2s finishes... Task_4s begins... Task_4s finishes...
```

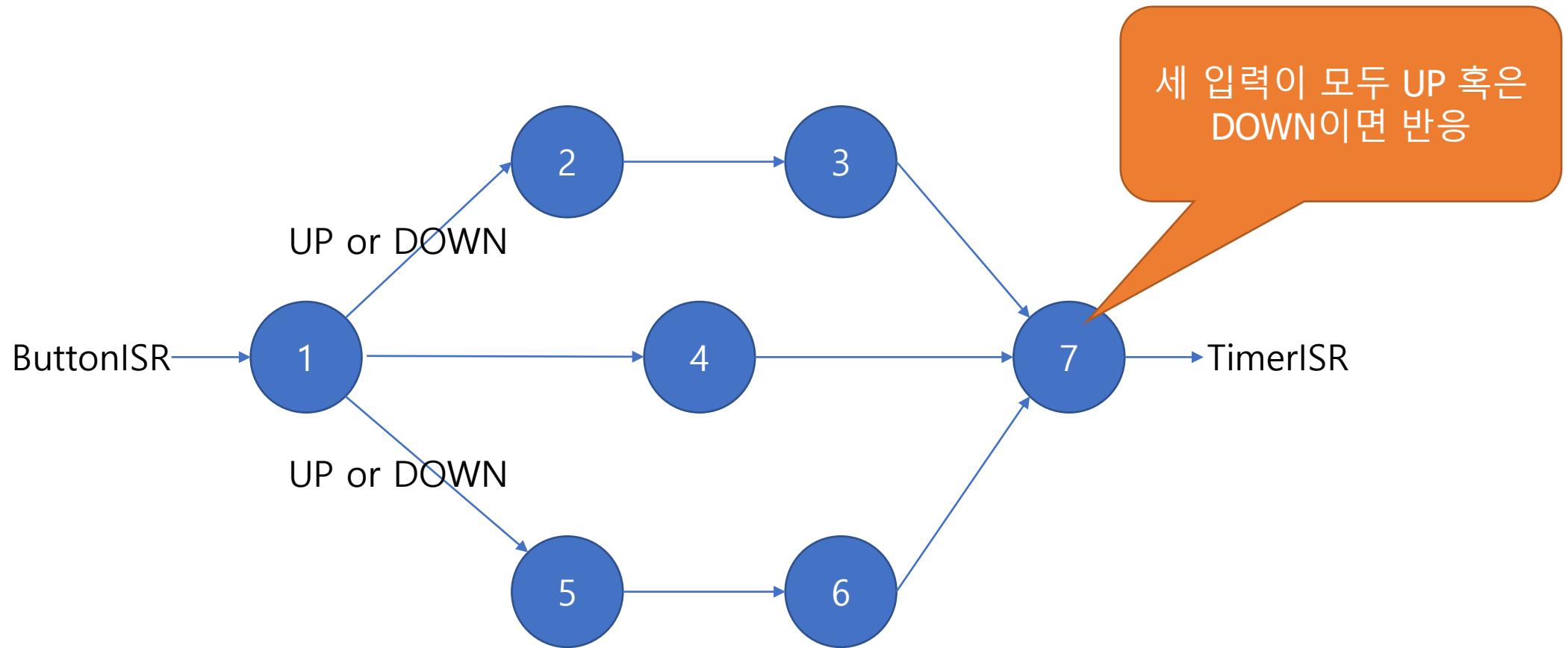

14. End-to-End Delay

- Runnable 순서에 따른 Reaction 지연 발생

```
0: [NA] Task_2s begins... Task_2s finishes... Task_4s begins... Task_4s finishes... Task_8s begins... Task_8s finishes... <BUTTON UP>
1: [NA]
2: [NA] Task_2s begins... Task_2s finishes...
3: [NA]
4: [NA] Task_2s begins... Task_2s finishes... Task_4s begins... Task_4s finishes...
5: [NA]
6: [NA] Task_2s begins... Task_2s finishes...
7: [NA]
8: [NA] Task_2s begins... Task_2s finishes... Task_4s begins... Task_4s finishes... Task_8s begins... Task_8s finishes...
9: [NA]
10: [NA] Task_2s begins... Task_2s finishes...
11: [NA]
12: [NA] Task_2s begins... Task_2s finishes... Task_4s begins... Task_4s finishes...
13: [NA]
14: [NA] Task_2s begins... Task_2s finishes...
15: [NA]
16: [NA] Task_2s begins... Task_2s finishes... Task_4s begins... Task_4s finishes... Task_8s begins... Task_8s finishes...
17: [NA]
18: [NA] Task_2s begins... Task_2s finishes...
19: [NA]
20: [NA] Task_2s begins... Task_2s finishes... Task_4s begins... Task_4s finishes...
21: [NA]
22: [NA] Task_2s begins... Task_2s finishes...
23: [NA]
24: [NA] Task_2s begins... Task_2s finishes... Task_4s begins... Task_4s finishes... Task_8s begins... Task_8s finishes...
25: [UP]
26: [UP] Task_2s begins... Task_2s finishes...
```

14. End-to-End Delay

- 아래 복잡한 DAG 구조를 정의하고 Delay 측정



Questions

